

实验 2-嵌入式软件单元测试实验

1. 实验介绍

本实验旨在通过构建单元测试环境，对一个嵌入式系统中的滤波模块进行功能与性能测试。实验分为两部分：一是在 PC 机环境下搭建单元测试框架，进行功能测试与周转时间性能测试；二是将测试环境移植到目标板端，通过串口通信实现黑盒测试。对比分析 PC 端与目标板端的测试结果，理解不同平台下程序执行的时间特性。

2. 实验目的

- (1) 掌握在 PC 端搭建嵌入式单元测试环境的方法，包括测试框架配置、用例设计与覆盖率分析。
- (2) 学会对嵌入式模块进行功能测试与性能测试，能够设计负载变化并记录程序执行时间。
- (3) 理解代码覆盖率概念，并能通过工具观察覆盖率结果。
- (4) 掌握在目标板实现相同测试的方法，熟悉跨平台时间获取函数的差异。
- (5) 通过黑盒测试分析程序的时间复杂度，理解负载对执行时间的影响。
- (6) 培养实验报告撰写能力与系统性调试技能。

3. 实验环境

(1) **硬件：**嵌入式软件测试教学实验系统板-A1、USB 口串口线、USB 口 ST-Link 烧录线、电源；Win 系统开发电脑（至少 2 个 USB 接口）

(2) 软件

- 代码编辑器：Keil MDK v535、Cursor/VScode Copilot（推荐辅助使用）
- 烧写工具：ST-Link
- 串口软件：XCOM V3.0.exe

- 其它工具：Python、Matlab 自动化脚本环境

4. 实验任务

4.1. 实验任务 1 — 环境搭建与单元测试框架配置

目标：在 PC 端搭建适用于嵌入式 C 代码的单元测试环境，配置测试框架与代码覆盖率工具，确保测试环境可正常运行。

验收标准：测试框架可成功编译并运行示例测试用例，覆盖率工具能正确生成覆盖率报告。

4.2. 实验任务 2 — 滤波模块功能测试与性能分析

目标：对提供的滤波模块进行功能测试与性能评估。设计多组测试用例（涵盖边界值、典型值、异常值），验证模块功能正确性；同时随机生成不同负载（如数据量、数据复杂度），记录程序执行时间，分析时间随负载变化的趋势。

验收标准：所有功能测试用例通过，代码覆盖率达到 100%；性能测试脚本能自动生成负载、记录执行时间，输出时间-负载曲线图，并完成时间复杂度趋势分析。

4.3. 实验任务 3 — 目标板测试与串口通信

目标：将滤波模块与测试代码移植到目标板，通过串口接收 PC 端发送的测试指令，执行测试并返回结果（包括功能测试结果与执行时间）。

验收标准：PC 端可通过串口发送指令控制目标板执行测试，目标板能正确返回测试数据与时间信息。

4.4. 实验任务 4 — 目标板三种方法时间性能测试

目标：在目标板上实现三种数据处理方法的时间性能测试（方法 A：寻找最奇特的数值；方法 B：数据聚合处理；方法 C：关联性分析），通过串口通信进行时间性能测试，分析各方法的执行时间特性，计算并验证理论时间复杂度。

方法 A：寻找最奇特的数值

- 输入：一个数组长度 n (1 到 1000)
- 功能：从系统数据其中 n 个数据中挑选最奇特的一个数据
- 例子：1 5 -> 最奇特的数值 = 19

方法 B：数据聚合处理

- 输入：一个处理范围 n (0 到 1000)
- 功能：对系统数据进行聚合运算，返回处理后的综合指标值
- 例子：2 100 -> 100 的聚合结果 = 15550

方法 C：关联性分析

- 输入：一个分析深度 n (0 到 300)
- 功能：执行数据关联度计算，生成综合关联指标 (Metric)
- 例子：3 50 -> 关联指标: 62475

验收标准：通过串口采集不同输入规模下的执行时间数据，绘制性能对比图，分析并验证三种方法的时间复杂度。

4.5. 实验任务 5 — 源码记录、复现与报告提交

目标：整理实验中修改的固件源码、PC 端脚本及实验报告。

验收标准：源码可编译运行，脚本能够执行实验流程，实验报告结构清晰、内容完整，包括实验步骤、数据分析和结论。

5. 案例 滑动滤波程序的动态单元性能测试

源代码

```
#include <stdio.h>
#define WINDOW_SIZE 5 // 滑动窗口的大小
// 滑动滤波函数
// data: 输入数据数组
// result: 滤波结果数组
// dataSize: 输入数据的大小
void slidingFilter(const int data[], int result[], int dataSize) {
    int sum = 0;
    // 初始化窗口的和
    for(int i = 0; i < WINDOW_SIZE; ++i) {
        sum += data[i];
        result[i] = sum / (i + 1);
    }
    // 对每个数据点进行滑动滤波处理
    for(int i = WINDOW_SIZE; i < dataSize; ++i) {
        sum += data[i] - data[i - WINDOW_SIZE];
        result[i] = sum / WINDOW_SIZE;
    }
}

int main() {
    // 示例数据
    int data[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int dataSize = sizeof(data) / sizeof(data[0]);
    int result[dataSize];
    slidingFilter(data, result, dataSize);
    // 打印滤波结果
    printf("Original data: ");
    for(int i = 0; i < dataSize; ++i) {
        printf("%d ", data[i]);
    }
    printf("\nFiltered data: ");
    for(int i = 0; i < dataSize; ++i) {
        printf("%d ", result[i]);
    }
    printf("\n");
    return 0;
}
```

针对函数的周转时间进行测试

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <time.h>
#define WINDOW_SIZE 5
void slidingFilter(const int data[], int result[], int dataSize) {
    if (dataSize < WINDOW_SIZE) {
        printf("Error: dataSize should be at least the size of the window.\n");
        return;
    }
    int sum = 0;
    // 预计算初始的滑动窗口和
    for (int i = 0; i < WINDOW_SIZE; ++i) {
        sum += data[i];
    }
    // 初始窗口的平均值
    for (int i = 0; i < WINDOW_SIZE; ++i) {
        result[i] = sum / WINDOW_SIZE;
    }
    // 滑动窗口计算
    for (int i = WINDOW_SIZE; i < dataSize; ++i) {
        sum += data[i] - data[i - WINDOW_SIZE];
        result[i] = sum / WINDOW_SIZE;
    }
}

void measureExecutionTime(int startSize, int endSize, int step, const char
*filename) {
    FILE *file = fopen(filename, "w");
    if (file == NULL) {
        printf("Error opening file!\n");
        return;
    }
    // 添加表头信息
    fprintf(file, "DataSize,ExecutionTime\n");
    for (int size = startSize; size <= endSize; size += step) {
        int *data = (int *)malloc(size * sizeof(int));
        int *result = (int *)malloc(size * sizeof(int));
        // 生成随机数据
        for (int i = 0; i < size; ++i) {
            data[i] = rand() % 100;
        }
        // 测量滑动滤波器的执行时间
        clock_t start = clock();
        slidingFilter(data, result, size);
        clock_t end = clock();
        uint32_t start = HAL_GetTick(); // 单位：毫秒

```

```

// ... 被测代码 ...
uint32_t elapsed_ms = HAL_GetTick() - start;
    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;
    fprintf(file, "%d, %f\n", size, time_spent);
    free(data);
    free(result);
}
fclose(file);
}
int main() {
    srand(time(NULL)); // 初始化随机数生成器种子
    measureExecutionTime(100, 10000, 100, "execution_times.csv");
    return 0;
}

```

1. 代码的解释

#include <stdio.h>: 包含标准输入输出库，用于文件操作、控制台打印等功能。

#include <stdlib.h>: 包含标准库，用于动态内存分配、随机数生成等功能。

#include <time.h>: 包含时间相关的函数，如测量代码执行时间、获取当前时间等。

#define WINDOW_SIZE 5: 定义滑动窗口的大小为 5。这个常量用于指定滑动窗口在数组中每次计算的元素个数。

2. slidingFilter 函数

输入参数:

data[]: 要进行滑动滤波的输入数组。

result[]: 存储滤波结果的输出数组。

dataSize: 输入数据数组的大小。

数据有效性检查:

if (dataSize < WINDOW_SIZE): 检查输入数据的大小是否至少为滑动窗口的大小。如果不满足条件，打印错误信息并返回，避免数组越界访问。

滑动窗口滤波过程:

初始窗口和计算：

for (int i = 0; i < WINDOW_SIZE; ++i): 对数组的前 WINDOW_SIZE 个元素进行累加计算，得到初始窗口和。

初始化结果数组：

for (int i = 0; i < WINDOW_SIZE; ++i): 将初始窗口的平均值赋给结果数组的前 WINDOW_SIZE 个元素。这样做是为了初始化这部分值，避免后续操作中无法填充这些位置。

滑动窗口滤波：

for (int i = WINDOW_SIZE; i < dataSize; ++i): 从第 WINDOW_SIZE 个元素开始，滑动窗口每次将一个新的元素加到窗口和中，同时减去窗口中移除的元素，从而得到新的窗口和，并计算平均值存入结果数组。

3. measureExecutionTime 函数

输入参数：

startSize: 起始的数据大小。

endSize: 结束的数据大小。

step: 数据大小的步长，每次增加的数据大小。

filename: 输出结果的 CSV 文件名。

文件操作：

fopen(filename, "w"): 以写模式打开文件。如果文件打开失败（例如文件路径不存在），打印错误信息并返回。

fprintf(file, "DataSize,ExecutionTime\n"): 在文件开头写入 CSV 格式的表头，便于后续分析。

执行时间测量：

循环遍历不同的数据大小，从 startSize 到 endSize，每次增加 step。

malloc 分配大小为 size 的内存，用于存储随机生成的数据和滤波结果。

rand() 生成 size 个随机数，填充 data 数组。

使用 `clock()` 函数获取当前时间，在调用 `slidingFilter` 前后分别记录时间，计算函数执行的时间差。

将数据大小和对应的执行时间以 CSV 格式写入文件。

内存释放：

使用 `free(data)` 和 `free(result)` 释放之前分配的内存，避免内存泄漏。

关闭文件：

`fclose(file)`：在所有操作完成后关闭文件，确保数据写入。

`main` 函数

随机数种子初始化：

`srand(time(NULL))`：使用当前时间初始化随机数生成器，确保每次运行时生成的随机数序列不同。

调用性能测试函数：

`measureExecutionTime(100, 10000, 100, "execution_times.csv")`：测试从数据大小 100 到 10000，步长为 100 的情况下，滑动窗口滤波器的执行时间，并将结果保存到 `execution_times.csv` 文件中。

程序结束：

`return 0;`：程序正常结束，返回值为 0。

总结：这个代码通过一个滑动窗口滤波器，计算不同数据规模下的平均值，并测量其执行时间。重构后的版本优化了初始窗口的计算方式，减少了不必要的计算，并增加了结果文件的可读性。通过动态分配内存和测量执行时间，该代码可以用于分析滑动窗口滤波器在不同数据规模下的性能表现。

```
import pandas as pd
import matplotlib.pyplot as plt
import os
# 文件路径
csv_file = '/Users/yichenwang/ESPT-Case/output/execution_times.csv'
# 确保文件存在
if not os.path.exists(csv_file):
    print(f"文件 {csv_file} 不存在，请检查文件路径或生成文件。")
```



```
        exit()
# 读取 CSV 文件
data = pd.read_csv(csv_file, header=None, names=['Size', 'Time'])
# 绘制图形
plt.figure(figsize=(10, 6))
plt.plot(data['Size'], data['Time'], marker='o')
plt.title('Execution Time vs. Data Size')
plt.xlabel('Data Size')
plt.ylabel('Execution Time (seconds)')
plt.grid(True)
plt.savefig('execution_time_plot.png')
plt.show()
```